

UNIVERSITY OF SASKATCHEWAN
Department of Computer Science

CMPT 215.3 MIDTERM EXAMINATION

March 5th, 2025

Total Marks: 50

CLOSED BOOK and CLOSED NOTES
no electronic devices

Time: 80 minutes

Instructions

Read each question carefully and write your answer legibly on the examination paper. **No other paper will be accepted.** You may use the backs of pages for rough work but all final answers must be in the spaces provided. The marks for each question are as indicated. Allocate your time accordingly.

Ensure that your name AND student number are clearly written on the examination paper and that your name is on every page.

Note: three RISC-V Reference pages are attached at the end of the exam.

Question	Marks
1 (5 marks)	
2 (5 marks)	
3 (16 marks)	
4 (18 marks)	
5 (6 marks)	
Total	

Name: _____

Student Number: _____

Student Number: _____

1. **General** (5 marks in total – 1 mark for each part) Give the technical term that best fits each of the following descriptions or definitions.

- (a) A data structure created during the assembly process that stores for each label in a program the memory address to which it corresponds.
- (b) Assembly language instructions that have no directly corresponding machine language instruction; an example in RISC-V assembly language is “li”.
- (c) A memory element storing a single bit, the value of which is changed only on a clock edge.
- (d) With the IEEE 754 floating point standard, the result of dividing 0 by 0.
- (e) An organization that has developed a variety of widely-used benchmark suites, including ones for assessing CPU performance.

2. **Computer Performance** (5 marks in total)

- (a) (1 mark) For a particular program as run on a particular system, 400 million instructions are executed, the clock cycle time is 0.25 nanoseconds, and the CPI is 2.

The CPU execution time is _____ seconds. (Fill in the blank.)

- (b) (1 mark) Suppose that a processor is modified to include a complex machine language instruction for some arithmetic operation (e.g. square root) that previously was implemented in software, i.e. using a sequence of simpler machine language instructions. Would you expect the MIPS rating for a program doing such operations to **increase**, to **decrease**, or to **not change**, with this modification?
- (c) (1 mark) A program is run on a system with only two types of machine language instructions: type A instructions each require 5 clock cycles, while type B instructions each require 2 clock cycles. Give an expression for the CPI as a function of the fraction f of the total instructions executed that are of type A.

Student Number: _____

- (d) (2 marks) Suppose that we can enhance a computer to make floating point instructions run three times faster than before. Give an expression for the speedup (ratio of **old to new** execution time) as a function of the fraction f of the old execution time spent on executing floating point instructions.

3. Computer Arithmetic and Logic Design (16 marks in total)

- (a) (4 marks) Perform the following conversions:

- (i) 70707_8 to base 2.
- (ii) 0.25_{10} to base 2.
- (iii) -6_{10} to 8 bit 2's complement.
- (iv) 22_3 to octal.

- (b) (3 marks) Give a logic equation in sum-of-products form for the logic function F specified by the following truth table. Simplify your logic equation to the maximum extent possible.

A	B	C	$F(A, B, C)$
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0

Name: _____

Page 4

Student Number: _____

(c) (2 marks) Give a truth table for a 2 data input, 1 selector input multiplexor.

(d) (2 marks) Give a diagram showing how a ripple carry adder capable of adding two 4 bit binary numbers $a_3a_2a_1a_0$ and $b_3b_2b_1b_0$ can be constructed from 1 bit full adders. You should clearly show the inputs and outputs of each 1 bit full adder and the connections between them.

(e) (2 marks) Suppose that we wished to implement in the ALU **signed overflow** detection for 32 bit integer addition operations. Give a logic function of the high order bits of the operands a and b (a_{31} and b_{31}) and the high order bit of the computed result c (c_{31}), in sum-of-products form, that has value 1 if signed overflow has occurred and has value 0 otherwise.

Student Number: _____

- (f) (2 marks) Consider the addition of 4 bit values $a_3a_2a_1a_0$ and $b_3b_2b_1b_0$. Give a logic function for the “carry-in” to the most significant bit position (i.e., the “carry-in” that will be added in when summing a_3 and b_3), using quantities $g_i = a_i \cdot b_i$, $p_i = a_i + b_i$, and the “carry-in” to the least significant bit position CarryIn_0 .
- (g) (1 mark) The IEEE 754 floating point standard allows some numbers to be represented in unnormalized form (called denormalized numbers, or “denorms”). Give one important benefit of this.

4. Machine and Assembly Language (18 marks in total)

- (a) (4 marks) Give a sequence of RISC-V assembly language instructions that calls a procedure “P” with its two arguments being 1) the **contents** of a memory word whose address is in register s1 and 2) the **address** of a memory word allocated using “A: .space 4”, and then stores the return value from “P” into the memory word whose address is in register s1. Your code must use the standard procedure call conventions. (You do not need to show any code for the procedure “P”.)
- (b) (2 marks) One of the tasks performed by a linker is that of patching the addressing fields of some instructions. Explain the need for this task.
- (c) (1 mark) Give a single RISC-V **RV32I base** integer instruction that could be used to multiply the integer contents of s0 by eight, storing the result back into s0.

Student Number: _____

(d) (1 mark) Suppose that first the contents of register a0 were pushed onto the stack, and then the contents of register ra. Assuming that no further changes to the stack have occurred, give a single RISC-V assembly language instruction that would restore the saved contents of register a0 (without removing from the stack).

(e) (3 marks) Translate the following code into an equivalent sequence of RISC-V assembly language instructions, assuming that registers s0 and s1 correspond to the integer variables i and j , respectively.

```
if (i > 4) then i = i - 1;
           else i = i + 1;
j = i + j;
```

(f) (5 marks) Below is RISC-V code that is entered with s0 containing the starting address of a first zero-terminated string and s1 containing the starting address of a second zero-terminated string. When a jump or branch is made to “done”, s2 should contain 1 if all the characters in the **second** string also appear at least once in the **first** string, and otherwise should contain 0. For example, if the first string is “eat a plum” and the second is “apple”, s2 should contain 1, while for “eat a peach” and “papaya”, s2 should contain 0. Fill in the missing instructions.

```
                li s2, 1
loop:           _____
                _____
                beqz t1, done
                mv t2, s0
loop2:         _____
                beq t3, t1, loop
                beqz t3, no
                _____
                _____
no:             mv s2, zero
                j done
```

Name: _____

Page 7

Student Number: _____

- (g) (2 marks) Consider a RISC-V branch instruction located at address 32100_{16} (in hexadecimal). Give the highest possible memory address of the branch target, in hexadecimal.

5. Datapath and Control (6 marks in total)

- (a) (3 marks) In the single clock cycle datapath discussed in class, there are **two** adders, separate from the ALU. What is each adder used for, and why can't these functions be performed using the ALU?

- (b) (2 marks) The single clock cycle datapath discussed in class includes several multiplexors. Describe the precise use of **one** of these.

- (c) (1 mark) In the single clock cycle datapath discussed in class, one of the inputs to the data memory component is "write data". What other input is needed?

The End

RISC-V REFERENCE

RISC-V Instruction Set

Core Instruction Formats

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12:10:5]				rs2		rs1		funct3		imm[4:1:11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20:10:1:11:19:12]										rd		opcode		J-type

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srlr	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 ≥ rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 ≥ rs2) PC += imm	zero-extends
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	
ecall	Environment Call	I	1110011	0x0	imm=0x0	Transfer control to OS	
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Transfer control to debugger	

Pseudo Instructions

Pseudoinstruction	Base Instruction(s)	Meaning
la rd, symbol	auipc rd, symbol[31:12] addi rd, rd, symbol[11:0]	Load address
l{b h w d} rd, symbol	auipc rd, symbol[31:12] l{b h w d} rd, symbol[11:0](rd)	Load global
s{b h w d} rd, symbol, rt	auipc rt, symbol[31:12] s{b h w d} rd, symbol[11:0](rt)	Store global
fl{w d} rd, symbol, rt	auipc rt, symbol[31:12] fl{w d} rd, symbol[11:0](rt)	Floating-point load global
fs{w d} rd, symbol, rt	auipc rt, symbol[31:12] fs{w d} rd, symbol[11:0](rt)	Floating-point store global
nop	addi x0, x0, 0	No operation
li rd, immediate	<i>Myriad sequences</i>	Load immediate
mv rd, rs	addi rd, rs, 0	Copy register
not rd, rs	xori rd, rs, -1	One's complement
neg rd, rs	sub rd, x0, rs	Two's complement
negw rd, rs	subw rd, x0, rs	Two's complement word
sext.w rd, rs	addiw rd, rs, 0	Sign extend word
seqz rd, rs	sltiu rd, rs, 1	Set if = zero
snez rd, rs	sltu rd, x0, rs	Set if $\neq$ zero
sltz rd, rs	slt rd, rs, x0	Set if < zero
sgtz rd, rs	slt rd, x0, rs	Set if > zero
fmv.s rd, rs	fsgnj.s rd, rs, rs	Copy single-precision register
fabs.s rd, rs	fsgnjx.s rd, rs, rs	Single-precision absolute value
fneg.s rd, rs	fsgnjn.s rd, rs, rs	Single-precision negate
fmv.d rd, rs	fsgnj.d rd, rs, rs	Copy double-precision register
fabs.d rd, rs	fsgnjx.d rd, rs, rs	Double-precision absolute value
fneg.d rd, rs	fsgnjn.d rd, rs, rs	Double-precision negate
beqz rs, offset	beq rs, x0, offset	Branch if = zero
bnez rs, offset	bne rs, x0, offset	Branch if $\neq$ zero
blez rs, offset	bge x0, rs, offset	Branch if $\leq$ zero
bgez rs, offset	bge rs, x0, offset	Branch if $\geq$ zero
bltz rs, offset	blt rs, x0, offset	Branch if < zero
bgtz rs, offset	blt x0, rs, offset	Branch if > zero
bgt rs, rt, offset	blt rt, rs, offset	Branch if >
ble rs, rt, offset	bge rt, rs, offset	Branch if $\leq$
bgtu rs, rt, offset	bltu rt, rs, offset	Branch if >, unsigned
bleu rs, rt, offset	bgeu rt, rs, offset	Branch if $\leq$, unsigned
j offset	jal x0, offset	Jump
jal offset	jal x1, offset	Jump and link
jr rs	jalr x0, rs, 0	Jump register
jalr rs	jalr x1, rs, 0	Jump and link register
ret	jalr x0, x1, 0	Return from subroutine
call offset	auipc x1, offset[31:12] jalr x1, x1, offset[11:0]	Call far-away subroutine
tail offset	auipc x6, offset[31:12] jalr x0, x6, offset[11:0]	Tail call far-away subroutine
fence	fence iorw, iorw	Fence on all memory and I/O

Registers

Register	ABI Name	Description	Saver
x0	zero	Zero constant	—
x1	ra	Return address	Callee
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0 / fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Fn args/return values	Caller
x12-x17	a2-a7	Fn args	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-7	ft0-7	FP temporaries	Caller
f8-9	fs0-1	FP saved registers	Callee
f10-11	fa0-1	FP args/return values	Caller
f12-17	fa2-7	FP args	Caller
f18-27	fs2-11	FP saved registers	Callee
f28-31	ft8-11	FP temporaries	Caller